VileKingdoms API

Version 1.0.5

The API of VileKingdoms lets the addons read the kingdoms, the towns, the structures and the wars of the server, and react to everything that happens to them through Bukkit events. Most actions have an event called **before**, that an addon can cancel, and an event called **after**, to react to what has been done.

The API is the only part of VileKingdoms an addon may compile against: it is made of interfaces, enums and events, published as the `VileKingdoms-API` artifact. The code of the plugin itself is licensed and must not be used.

How the plugin works, its commands and its configuration are explained in the server guide.

1. Adding the dependency

The plugin includes the API: the addon must not shade it, only compile against it.

The API is published on JitPack, from the public repository SirImperivm/VileKingdoms-API, with its sources and javadoc jars. The jars are also attached to the releases of that repository.

Gradle

```
repositories {
    maven { url = "https://jitpack.io" }
}

dependencies {
    compileOnly "com.github.SirImperivm:VileKingdoms-API:1.0.5"
}
```

Maven

```
<repositories>
    <repository>
        <id>jitpack.io</id>
        <url>https://jitpack.io</url>
    </repository>
</repositories>

<dependency>
    <groupId>com.github.SirImperivm</groupId>
    <artifactId>VileKingdoms-API</artifactId>
    <version>1.0.5</version>
    <scope>provided</scope>
</dependency>
```

Depending on the plugin

The addon must be enabled after VileKingdoms. In a `plugin.yml`:

```
depend: [VileKingdoms]        # or softdepend, when the addon works without it
```

In a `paper-plugin.yml`:

```
dependencies:
  server:
    VileKingdoms:
      load: BEFORE
      required: true
```

2. Getting the API

```
VileKingdomsAPI api = VileKingdomsAPI.get();
```

`VileKingdomsAPI.get()` throws an `IllegalStateException` when VileKingdoms is not enabled. The API is also a service of Bukkit:

```
RegisteredServiceProvider<VileKingdomsAPI> provider =
Bukkit.getServicesManager().getRegistration(VileKingdomsAPI.class);
VileKingdomsAPI api = provider == null ? null : provider.getProvider();
```

The kingdoms are loaded from the database a little after the plugin is enabled: until `api.isLoaded()` is true the lookups are empty.

Rules

- Every method of the API must be called from the main thread of the server, and every event is called from it.
- The models (`Kingdom`, `Town`, `Structure`, `War`) are **views**: they always show the current state of the object, and they are read only. Do not keep them for long: look the object up again by its id when you need it later.
- A view can refer to an object that does not exist anymore (e.g. a town released in the meantime): the getters returning other objects, such as `Town.getKingdom()`, are `@Nullable` for this reason.

3. The entry point: `VileKingdomsAPI`

Method	Returns
`getVersion()`	the version of the plugin
`isLoaded()`	whether the kingdoms and the towns have been loaded

Method	Returns
`getKingdoms()`	every kingdom
`getKingdom(UUID)`, `getKingdom(String)`	the kingdom by id, or by name or display name
`getPrimaryKingdom(UUID player)`	the kingdom of the town the player manages, otherwise the one they rule, otherwise the first by name
`getTowns()`	every town
`getTown(UUID)`, `getTown(String)`	the town by id, or by name or display name
`getLandSize()`	the side of the lands in blocks, the unit of the claims: it never changes once the first town is founded
`getTownAt(Location)`	the town owning the land of the location
`getTownsOf(UUID player)`	the towns the player is a member of
`getWarOf(Town)`	the war in progress of the town
`getActiveWars()`	the wars in progress
`isWarPeriod()`	whether the wars can be declared now
`getPeriodEndsAt()`	when the current period ends, `Long.MAX_VALUE` for a permanent peace

The lookups return an `Optional`, empty when the object does not exist.

4. The models

Kingdom

`getId()`, `getName()`, `getDisplayName()`, `getDescription()`, `getKingId()`, `getMembers()`, `getTowns()`, `getCapital()`, `getRelation(Kingdom)`, `isFriendlyFire()`, and the sums of its towns: `getEmeralds()`, `getWorth()`, `getBank()`, `getClaimsCount()`.

Town

`getId()`, `getName()`, `getDisplayName()`, `getKingdom()`, `getRank()`, `getStyle()`, `getMembers()`, `isMember(UUID)`, `getRoleName(UUID)`, `getClaims()`, `hasClaim(LandPosition)`, `getBank()`, `getEmeralds()` (kept by the town hall), `getEmeraldCapacity()`, `getWorth()`, `getImmuneUntil()`, `isDestroyed()`, `getStructures()`, `getSpawn()`.

Structure

`getId()`, `getType()`, `getStyle()`, `getLevel()`, `getDefenseLevel()`, `getTown()`, `getLocation()`, `getEmeralds()` (the emeralds of the town for the town hall, 0 for the others), `getLinkedCollector()` (of an extractor, null when it is not linked).

War

`getId()`, `getAttackerId()`, `getDefenderId()`, `getAttacker()`, `getDefender()`, `getDeclarerId()`, `getStake()`, `getSquad()`, `getStartedAt()`, `getEndedAt()`, `getOutcome()`, `getContestedLands()`.

Values

Type	Values
`RelationType`	`ALLY`, `NEUTRAL`, `ENEMY`
`StructureType`	`TOWN_HALL`, `EMERALD_EXTRACTOR`, `EMERALD_COLLECTOR`, `RESIDENCE`, `DEPOT`
`WarOutcome`	`NONE` (in progress), `ATTACKER_WON`, `DEFENDER_WON`, `ABANDONED`, `DRAW`
`LeaveReason`	`LEFT`, `KICKED`, `BANNED`, `REMOVED_BY_ADMIN`, `TAX_NOT_PAID`, `ENEMY_CONFLICT`, `TOWN_RELEASED`
`LandPosition`	record of `world`, `x`, `z` of a land (the block coordinates divided by the side of the lands), with `LandPosition.of(Location)`, `getMinBlockX()` and `getMinBlockZ()`

5. Events

Every event extends `VileKingdomsEvent`, a Bukkit `Event`; the events called before an action extend `CancellableVileKingdomsEvent` and can be cancelled. They are listened to like any Bukkit event:

```
public final class ClaimListener implements Listener {

    @EventHandler(ignoreCancelled = true)
    public void onPreClaim(TownPreClaimEvent event) {
        if (event.getLand().world().equals("spawn")) {
            event.setCancelled(true);
            if (event.getPlayer() != null) event.getPlayer().sendMessage("You cannot claim in
the spawn world.");
        }
    }

    @EventHandler
    public void onClaim(TownClaimEvent event) {
        Bukkit.getLogger().info(event.getTown().getName() + " claimed " + event.getLand());
    }
}
```

Cancelling an event called before an action of a player stops the action without any message of VileKingdoms: the addon tells the player why. The server owner can still configure a message in `kingdom-errors.cancelled-by-addon` of the locales, empty by default.

Some actions have only the event after them: the releases of every claim of a town (`/town admin release-all-for`, the claims over the limit of a lost rank) call a `TownUnclaimEvent` for each land, and the members kicked by the conquest of a town call a `TownMemberLeaveEvent`.

Kingdoms

Event	Cancellable	When	Getters
`KingdomCreateEvent`	no	Called when a kingdom has been created.	`getKingdom()` Kingdom: the new kingdom `getCreator()` Player: its king
`KingdomDisbandEvent`	no	Called when a kingdom has been disbanded or destroyed: it does not exist anymore.	`getKingdomId()` UUID: the id the kingdom had `getName()` String: the name the kingdom had `isAdmin()` boolean: whether an admin destroyed it
`KingdomPreCreateEvent`	yes	Called before a player creates a kingdom.	`getCreator()` Player: the player creating the kingdom, its king `getDisplayName()` String: the display name chosen
`KingdomPreDisbandEvent`	yes	Called before a kingdom is disbanded by its king or destroyed by an admin.	`getKingdom()` Kingdom: the kingdom `getActor()` CommandSender: who disbands it `isAdmin()` boolean: whether an admin destroys it
`KingdomPreRenameEvent`	yes	Called before the king renames the kingdom.	`getKingdom()` Kingdom: the kingdom `getPlayer()` Player: the player making the action `getNewDisplayName()` String: the new display name
`KingdomRelationChangeEvent`	no	Called when the relation between two kingdoms has changed.	`getKingdom()` Kingdom: the kingdom `getOther()` Kingdom: the other kingdom `getOldRelation()` RelationType: the relation they had `getNewRelation()` RelationType: their new relation
`KingdomRenameEvent`	no	Called when a kingdom has been renamed.	`getKingdom()` Kingdom: the kingdom, with its new name `getOldDisplayName()` String: the display name it had

Towns

Event	Cancellable	When	Getters
`TownCapitalChangeEvent`	no	Called when the king has chosen a new capital.	`getKingdom()` Kingdom: the kingdom `getOldCapital()` Town: the capital it had `getNewCapital()` Town: the new capital

Event	Cancellable	When	Getters
`TownClaimEvent`	no	Called when a land has been claimed for a town.	`getTown()` Town: the town `getLand()` LandPosition: the land `getPlayer()` Player: who claimed it
`TownEmeraldConvertEvent`	no	Called when real emeralds have been converted into emeralds of a town.	`getTown()` Town: the town `getPlayer()` Player: the player who made the action `getAmount()` double: the emeralds the town received
`TownFoundEvent`	no	Called when a town has been founded.	`getTown()` Town: the new town `getFounder()` Player: its first leader
`TownKingdomChangeEvent`	no	Called when a town now belongs to another kingdom, conquered in a war.	`getTown()` Town: the town `getOldKingdomId()` UUID: the kingdom it belonged to `getNewKingdom()` Kingdom: its new kingdom
`TownMemberJoinEvent`	no	Called when a player has joined a town.	`getTown()` Town: the town `getMember()` UUID: the new member
`TownMemberLeaveEvent`	no	Called when a player is not a member of a town anymore.	`getTown()` Town: the town `getMember()` UUID: the former member `getReason()` LeaveReason: why
`TownMemberPreJoinEvent`	yes	Called before a player joins a town.	`getTown()` Town: the town `getPlayer()` Player: the player joining it
`TownPreClaimEvent`	yes	Called before a land is claimed for a town, by a member or by an admin.	`getTown()` Town: the town `getLand()` LandPosition: the land `getPlayer()` Player: who claims it
`TownPreEmeraldConvertEvent`	yes	Called before real emeralds are converted into emeralds of a town.	`getTown()` Town: the town `getPlayer()` Player: the player making the action `getAmount()` double: the emeralds the town receives
`TownPreFoundEvent`	yes	Called before a player founds a town for their kingdom.	`getFounder()` Player: the founder, its first leader `getKingdom()` Kingdom: the kingdom of the town `getDisplayName()` String: the display name chosen `getLand()` LandPosition: the land of the town hall

Event	Cancellable	When	Getters
`TownPreRankUpEvent`	yes	Called before a town reaches the next rank.	`getTown()` Town: the town `getPlayer()` Player: the player making the action `getNewRank()` int: the rank it reaches
`TownPreReleaseEvent`	yes	Called before a town is released by its members or by an admin.	`getTown()` Town: the town `getActor()` CommandSender: who releases it
`TownPreUnclaimEvent`	yes	Called before a land of a town is released by a member or by an admin.	`getTown()` Town: the town `getLand()` LandPosition: the land `getPlayer()` Player: who releases it
`TownRankChangeEvent`	no	Called when the rank of a town has changed: ranked up, or lost.	`getTown()` Town: the town, with its new rank `getOldRank()` int: the rank it had
`TownReleaseEvent`	no	Called when a town has been released: it does not exist anymore.	`getTownId()` UUID: the id the town had `getName()` String: the name the town had `getKingdomId()` UUID: the kingdom the town belonged to
`TownUnclaimEvent`	no	Called when a land of a town has been released.	`getTown()` Town: the town `getLand()` LandPosition: the land `getPlayer()` Player: who released it

Structures and defenses

Event	Cancellable	When	Getters
`DefenderPreSpawnEvent`	yes	Called before the defenders of a structure appear to attack an enemy.	`getStructure()` Structure: the defended structure `getTarget()` Player: the enemy they attack `getCount()` int: how many defenders appear
`ExtractorLinkEvent`	no	Called when an extractor has been linked to a collector.	`getExtractor()` Structure: the extractor `getCollector()` Structure: the collector `getPlayer()` Player: the player who made the action

Event	Cancellable	When	Getters
`ExtractorPreLinkEvent`	yes	Called before an extractor is linked to a collector.	`getExtractor()` Structure: the extractor `getCollector()` Structure: the collector `getPlayer()` Player: the player making the action
`StructureDefensePreUpgradeEvent`	yes	Called before the defenses of a structure reach the next level.	`getStructure()` Structure: the structure `getPlayer()` Player: the player making the action `getNewLevel()` int: the defense level it reaches
`StructureDefenseUpgradeEvent`	no	Called when the defenses of a structure have reached the next level.	`getStructure()` Structure: the structure, with its new defense level `getPlayer()` Player: the player making the action
`StructurePickupEvent`	no	Called when a structure has been picked up: it is not placed anymore.	`getStructureId()` UUID: the id of the structure `getType()` StructureType: its type `getTown()` Town: the town it belonged to `getPlayer()` Player: the player making the action
`StructurePlaceEvent`	no	Called when a structure has been placed.	`getStructure()` Structure: the structure `getPlayer()` Player: the player making the action
`StructurePrePickupEvent`	yes	Called before a player picks up a structure into their inventory.	`getStructure()` Structure: the structure `getPlayer()` Player: the player making the action
`StructurePrePlaceEvent`	yes	Called before a player places a structure in a claim of their town.	`getTown()` Town: the town `getType()` StructureType: the type of the structure `getLevel()` int: its level `getLocation()` Location: the lowest corner of the structure at the ground `getPlayer()` Player: the player making the action
`StructurePreUpgradeEvent`	yes	Called before a structure reaches the next level.	`getStructure()` Structure: the structure `getPlayer()` Player: the player making the action `getNewLevel()` int: the level it reaches

Event	Cancellable	When	Getters
`StructureUpgradeEvent`	no	Called when a structure has reached the next level.	`getStructure()` Structure: the structure, with its new level `getOldLevel()` int: the level it had

Wars

Event	Cancellable	When	Getters
`WarDeclareEvent`	no	Called when a war has been declared and started.	`getWar()` War: the war
`WarEndEvent`	no	Called when a war has ended.	`getWar()` War: the war, with its outcome `getOutcome()` WarOutcome: the outcome `getPaid()` double: the money paid between the banks
`WarLandContestEvent`	no	Called when the attackers have contested a land of the defender.	`getWar()` War: the war `getLand()` LandPosition: the contested land
`WarPeriodEndEvent`	no	Called when a period of war has ended and a period of peace has started.	`isPermanentPeace()` boolean: whether the peace lasts until an admin starts a period of war
`WarPeriodStartEvent`	no	Called when a period of war has started: the wars can be declared.	`getEndsAt()` long: when the period ends, in epoch milliseconds
`WarPreDeclareEvent`	yes	Called before a town declares war on a town of an enemy kingdom.	`getAttacker()` Town: the attacking town `getDefender()` Town: the defending town `getDeclarer()` Player: the player declaring the war `getStake()` double: the stake of the war

6. Examples

Reading the town of a player

```
VileKingdomsAPI api = VileKingdomsAPI.get();
for (Town town : api.getTownsOf(player.getUniqueId()))
    player.sendMessage(town.getDisplayName() + ": rank " + town.getRank() + ", " +
town.getMembers().size() + " members");
```

Rewarding the winners of a war

```
@EventHandler
public void onWarEnd(WarEndEvent event) {
    if (event.getOutcome() != WarOutcome.ATTACKER_WON) return;

    for (UUID attacker : event.getWar().getSquad()) {
        Player player = Bukkit.getPlayer(attacker);
        if (player != null) player.getInventory().addItem(new ItemStack(Material.DIAMOND, 5));
    }
}
```

Limiting the kingdoms of a player

```
@EventHandler(ignoreCancelled = true)
public void onPreCreate(KingdomPreCreateEvent event) {
    if (!event.getCreator().hasPermission("myaddon.create-kingdom")) {
        event.setCancelled(true);
        event.getCreator().sendMessage("You need a rank to found a kingdom.");
    }
}
```

7. WorldGuard flags

With WorldGuard installed VileKingdoms registers these state flags, all allowed by default:

Flag	When denied
`kingdoms-enabled`	no `/kingdom` and `/town` command works in the region, but the admin ones
`towns-enabled`	no `/town` command works in the region, but the admin ones
`town-claims-enabled`	the lands touching the region, even in part, cannot be claimed nor released; the automatic claim and release stop entering them. The admin claims and releases, `/town admin release-all-for` included, still work